Scenario-Based SQL Questions – Set 3

Practical SQL scenarios • Questions and logic explanations

Scenario 1: Find Customers with High Transaction Frequency

Question:
Identify customers who have made more than 10 transactions in the last month.

Logic Explanation:

Use the sales table, which stores one row per transaction with customer_id and sale_date.

Filter transactions of the last month using WHERE sale_date >= CURDATE() - INTERVAL 1 MONTH.

Apply GROUP BY customer_id and use COUNT() to count the transactions of each customer.

Use HAVING COUNT(*) > 10 to keep only customers with more than 10 transactions.

Join with the customers table to display the customer name.

Concepts Used:

Date Functions (CURDATE(), INTERVAL)

Aggregation (COUNT())

Grouping (GROUP BY)

Filtering Groups (HAVING)

Scenario 2: Calculate Department-Wise Average Salary

Question:
Find the average salary of employees in each department.

Logic Explanation:

Perform an INNER JOIN between the employees table and the departments table using department_id.

Use the AVG() function to calculate the average salary.

Apply a GROUP BY clause on department_name to get one result per department.

Use ROUND() to display the average salary with two decimal places.

Concepts Used:

Joins (INNER JOIN)

Aggregate Functions (AVG())

Grouping (GROUP BY)

Scenario 3: Identify Top Spending Customers

Question:
Find the top 5 customers who have spent the most on purchases.

Logic Explanation:

Join the customers table with the sales table using customer_id.

Use the SUM() function to calculate the total amount spent by each customer.

Apply GROUP BY on customer_id and customer_name.

Use ORDER BY total_spent DESC so that the highest spenders come first.

Use LIMIT 5 to restrict the output to the top 5 customers.

Concepts Used:

Joins (INNER JOIN)

Aggregation (SUM())

Sorting (ORDER BY)

Limiting Results (LIMIT)

Scenario 4: Detect Data Gaps in Orders

Question:
Identify dates where no orders were placed.

Logic Explanation:

Generate a continuous list of dates between the first and last order date using a recursive CTE (WITH RECURSIVE).

Perform a LEFT JOIN between the generated date list and the orders table on order_date.

Filter records using WHERE orders.order_id IS NULL to find dates without any matching orders.

Display the missing dates in ascending order.

Concepts Used:

Recursive CTE (WITH RECURSIVE)

Joins (LEFT JOIN)

Filtering with NULL Values

Date Functions (MIN(), MAX(), INTERVAL)

Scenario 5: Identify Employees with Similar Salaries

Question:
Find pairs of employees working in the same department with a salary difference of less than 10%.

Logic Explanation:

Perform a self-join on the employees table using department_id so that each employee is compared with colleagues in the same department.

Use e1.employee_id < e2.employee_id to avoid matching an employee with themselves and to avoid duplicate pairs.

Calculate the salary difference using ABS(e1.salary - e2.salary).

Keep only pairs where the difference is less than 10% of the higher salary: ABS(e1.salary - e2.salary) < 0.10 * GREATEST(e1.salary, e2.salary).

Display both employee names, their salaries and the department.

Concepts Used:

Self Join

Mathematical Functions (ABS(), GREATEST())

Filtering (WHERE)